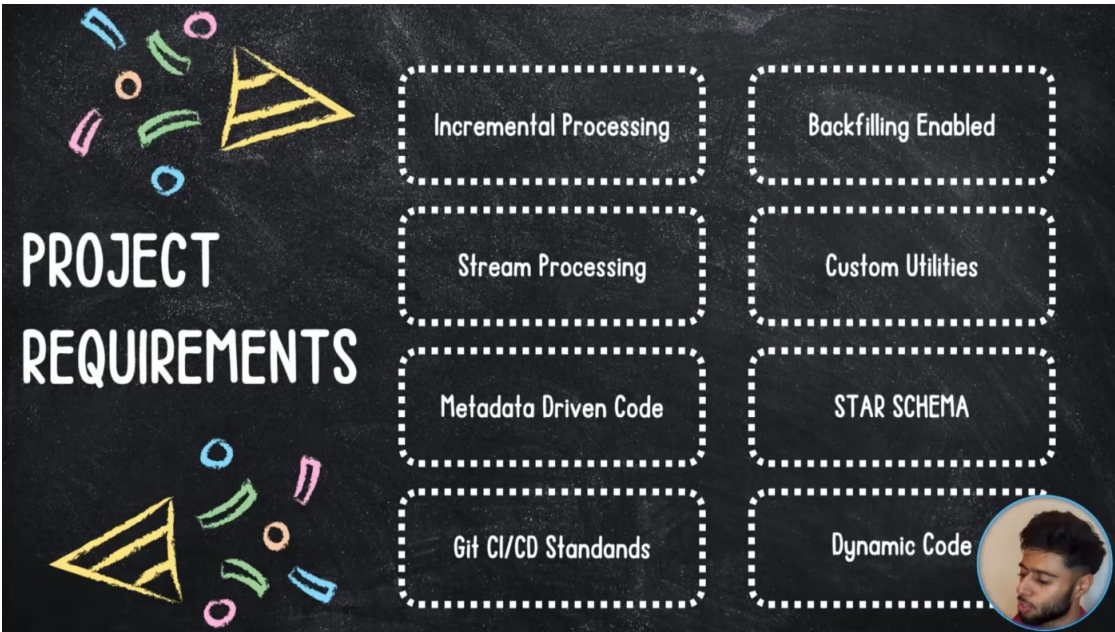


Azure END-TO-END DATA ENGINEERING PROJECT

Production-Ready Lakehouse Pipeline with Incremental Processing, SCD Type 2 & CI/CD



Project Scope & Key Requirements

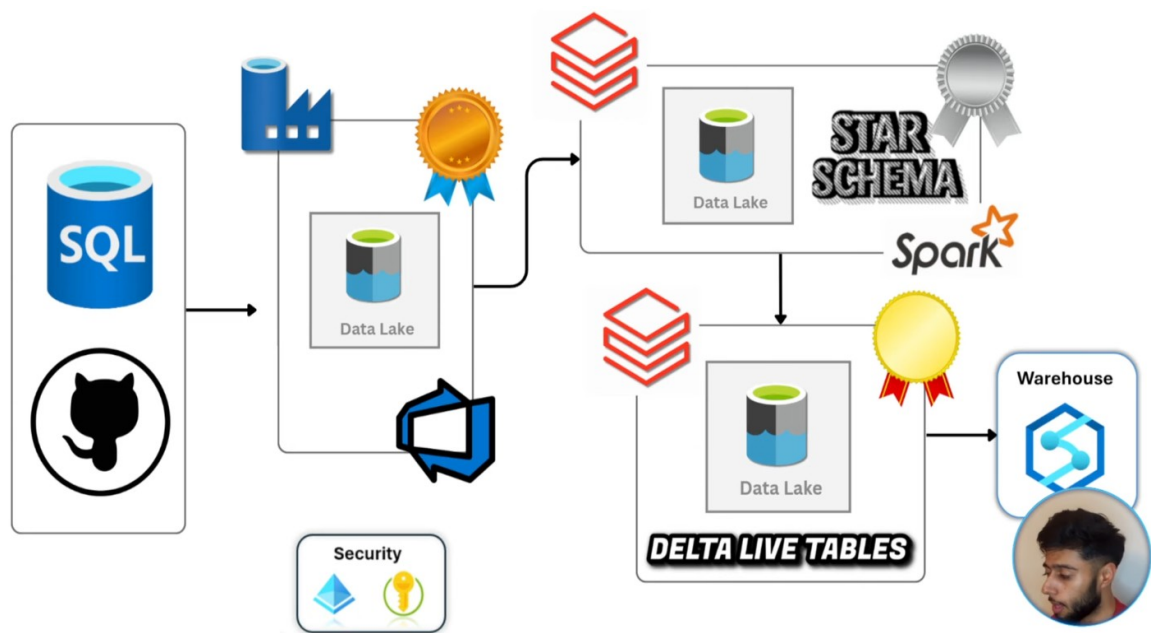
Project Type	End-to-End Data Engineering Portfolio Project
Domain	Big Data Engineering Cloud Data Platforms Lakehouse Architecture
Architecture	Medallion (Bronze-Silver-Gold) + Star Schema + SCD Type 2
Core Technologies	Azure Data Factory, Azure Databricks, PySpark, Unity Catalog, Delta Lake, DLT
Key Patterns	Incremental CDC, Watermarking, Metadata-Driven Pipelines, Declarative SCD
Governance	Unity Catalog with External Locations & Access Control

1. EXECUTIVE SUMMARY

This project demonstrates a complete, production-grade data engineering solution built on Microsoft Azure. I designed and implemented **an end-to-end lakehouse pipeline that ingests data from a transactional source, processes it through multiple quality layers, applies dimensional modeling with historical tracking, and deploys the entire solution using infrastructure-as-code practices.**

The solution handles real-world complexities including incremental data loading, backfilling, schema evolution, data quality enforcement, and automated CI/CD deployment — all while maintaining full governance through Unity Catalog.

2. ARCHITECTURE & DESIGN PRINCIPLES



End-to-End Azure Data Project Architecture

I adopted the **Medallion Architecture** combined with a **Star Schema** in the Gold layer and **SCD Type 2** for slowly changing dimensions. This design ensures data lineage, auditability, and the ability to reprocess historical data while maintaining business-level curated datasets for analytics and machine learning..

Design Principles Applied:

1. **Idempotency** — All pipelines can be safely re-run without duplicating data.
2. **Exactly-Once Processing** — Achieved through watermarking and Autoloader checkpoints.
3. **Metadata-Driven** — Dynamic SQL generation and parameterized pipelines for reusability.
4. **Governance-First** — Unity Catalog governs all access with fine-grained permissions.
5. **Infrastructure as Code** — Full CI/CD using Databricks Asset Bundles.

3. TECHNOLOGY STACK & JUSTIFICATION

Component	Technology Chosen	Rationale
Orchestration & Ingestion	Azure Data Factory	Native Azure integration, visual pipeline design, built-in watermark support, cost-effective for batch workloads
Source System	Azure SQL Database	Reliable transactional store with CDC capabilities for realistic source simulation
Data Lake Storage	Azure Data Lake Gen2	Hierarchical namespace, POSIX permissions, native integration with Databricks and ADF
Processing Engine	Azure Databricks (PySpark + Serverless)	Unified analytics platform, excellent Spark performance, native Delta Lake support, fast startup
Data Governance	Unity Catalog	Centralized governance, lineage,

		access control, and external table management
Incremental Loading	Autoloader + Delta Lake	Cloud-native streaming with exactly-once semantics, schema evolution, checkpointing
Dimensional Modeling	Delta Live Tables (Lakeflow)	Declarative SCD Type 2 with automatic CDC handling and data quality expectations
Dynamic Pipelines	Jinja2 + Python	Metadata-driven SQL generation for maintainable and scalable transformation logic
CI/CD & Deployment	Databricks Asset Bundles + Git	Version-controlled, environment-aware deployment with YAML-based configuration
Monitoring & Alerting	Azure Logic Apps	Event-driven failure notifications integrated with ADF pipeline runs

Key Highlights from Technology Choices:

- Native Azure integration with built-in watermark support and cost-effective batch processing
- Unified analytics platform with excellent Spark performance and native Delta Lake support
- Centralized governance, lineage, and fine-grained access control via Unity Catalog
- Declarative SCD Type 2 with automatic CDC handling and data quality expectations
- Version-controlled, environment-aware deployment using YAML-based Databricks Asset Bundles

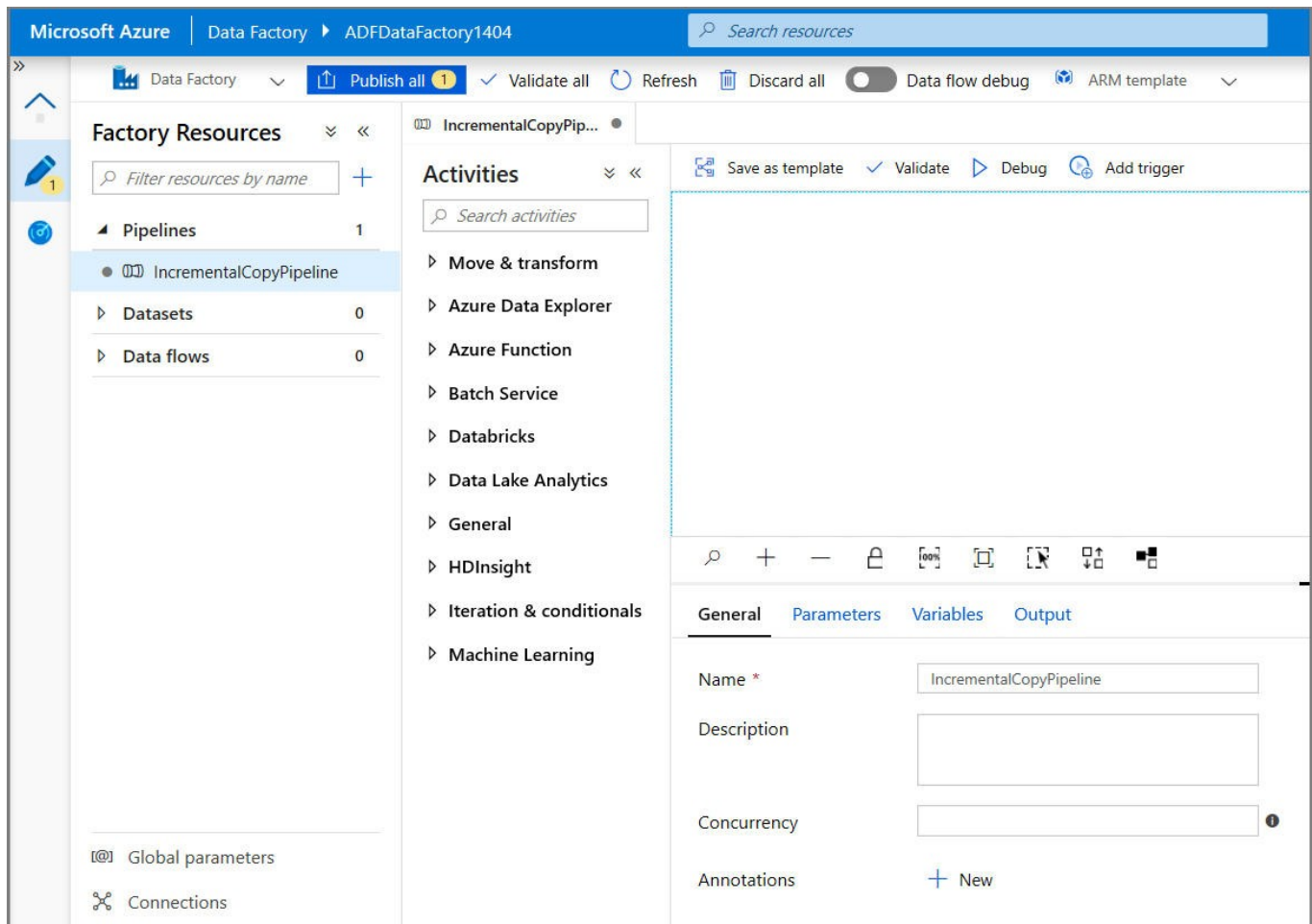
4. DETAILED IMPLEMENTATION

4.1 Incremental Ingestion Layer (Azure Data Factory)

I built a highly reusable, parameterized ingestion framework using Azure Data Factory. The pipeline uses a watermark pattern with a JSON control file stored in the Data Lake. Key components include:

- Lookup Activity to fetch the last processed CDC value from cdc.json
- Parameterized Copy Activity with dynamic query: `WHERE cdc_column > @last_cdc`
- Conditional logic to handle cases with no new data (avoid creating empty files)
- ForEach activity to process multiple tables using a single pipeline definition
- Backfilling capability through an optional from_date parameter

- Post-load watermark update using a separate Copy Activity



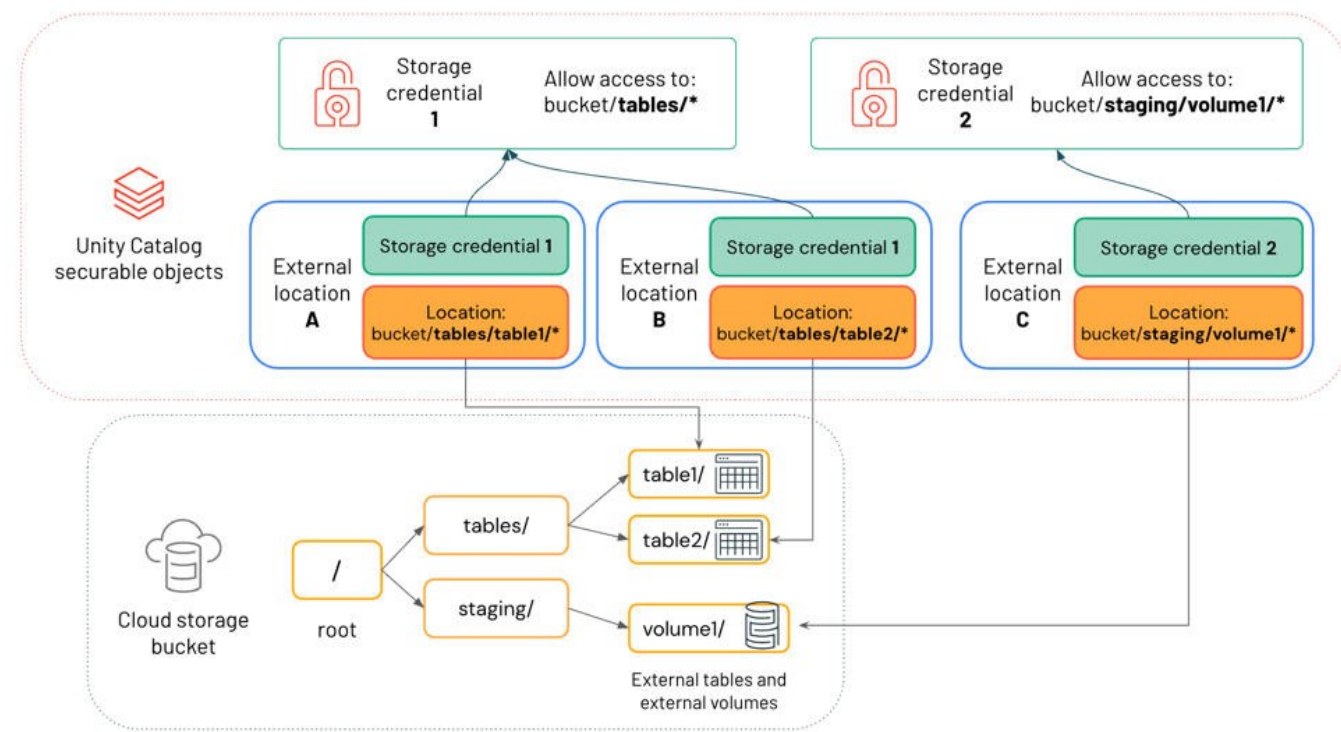
Azure Data Factory incremental Pipeline with Watermark & Parametrization

4.2 Bronze to Silver Processing (Databricks + Autoloader)

I implemented streaming ingestion from the Bronze layer using Autoloader (cloudFiles format). This provides ***exactly-once processing***, automatic schema inference/evolution, and checkpoint-based recovery. I created a modular Python utility package containing reusable transformation functions (column dropping, standardization, conditional flagging) that are imported across multiple notebooks.

4.3 Unity Catalog & Governance Layer

I configured a complete Unity Catalog setup including Metastore assignment, Access Connector for secure Data Lake access, External Locations, Catalogs, and Schemas (bronze, silver, gold). All tables are registered as external tables to maintain governance while allowing direct file access when needed.

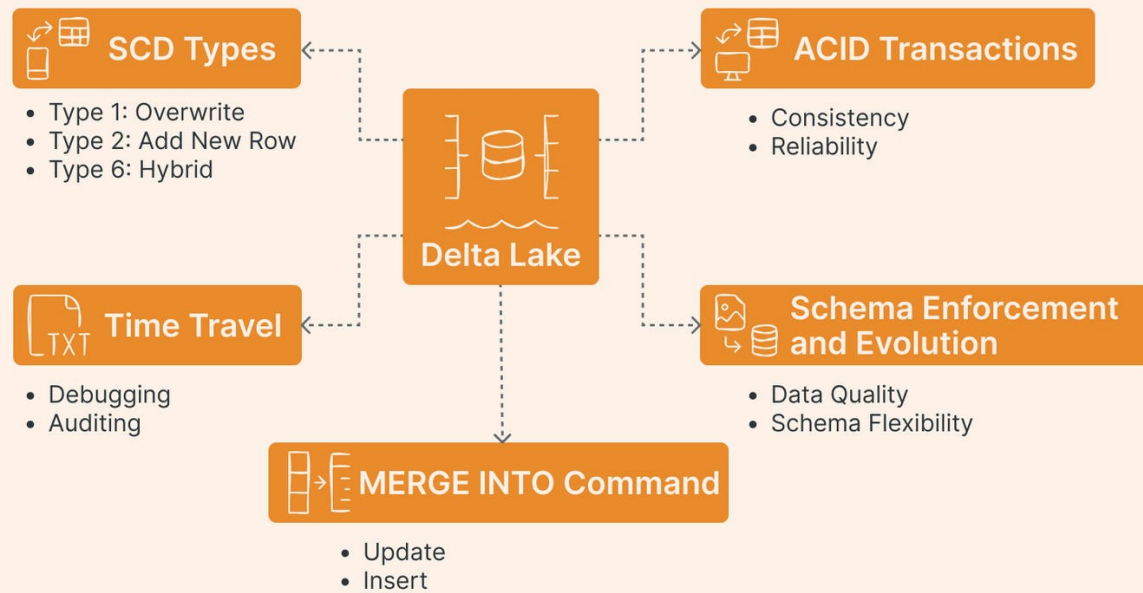


unity Catalog Structure with External locations and Securable Objects

4.4 Gold Layer – Star Schema & SCD Type 2

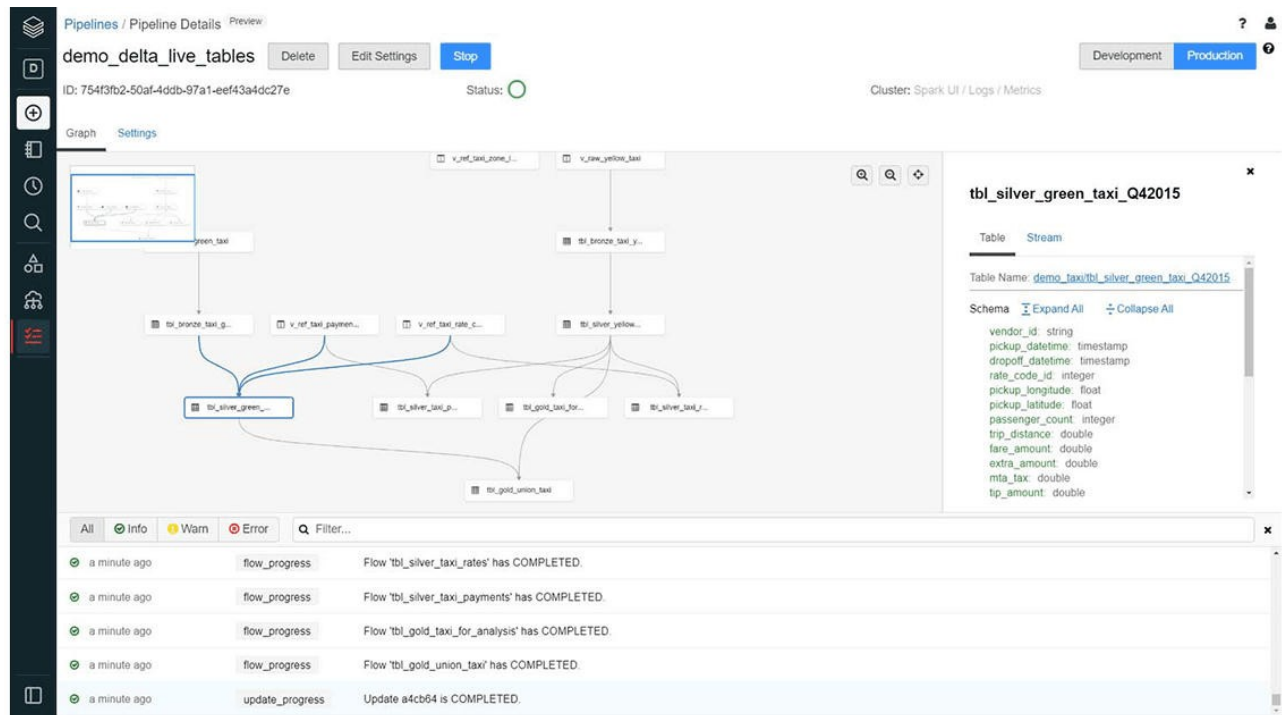
I designed and implemented a Star Schema in the Gold layer with fact and dimension tables. For dimensions requiring historical tracking (e.g., user profiles, track metadata), I **implemented SCD Type 2 using Delta Live Tables / Lakeflow Declarative Pipelines**. This approach uses automatic Change Data Capture flows with a `sequence_by` column (`updated_at`) to manage row versions, insert new records, update current records, and expire old versions — all declaratively.

Delta Lake for SCD Implementation



SCD type 2 Implementation using Delta Lake with start_date, end_date, and current_flag

Data quality is enforced using expectations (e.g., user_id IS NOT NULL) with options to warn, drop, or fail on violations.



Delta Live Tables Pipeline DAG showing lineage and execution monitoring

4.5 Metadata-Driven & Dynamic Pipelines

- To avoid hard-coded SQL for joins and business views, I implemented a metadata-driven approach using Jinja2 templating.
- This allows dynamic generation of SELECT statements, JOIN conditions, and column lists based on configuration metadata,
- making the solution highly maintainable and scalable to new tables.

4.6 CI/CD & Deployment

- I configured Databricks Asset Bundles using YAML definitions for notebooks, jobs, pipelines, and clusters. This enables
- version-controlled deployment across development and production environments with a single command, following
- infrastructure-as-code best practices.

5. Key Technical Achievements

- Built a fully parameterized, reusable ingestion framework handling 10+ tables through a single pipeline definition
- Achieved exactly-once incremental processing using watermark JSON + Autoloader checkpoints
- Implemented production-grade SCD Type 2 with automatic version management and data quality rules
- Established complete data governance using Unity Catalog with External Locations and fine-grained access control
- Created modular, importable Python transformation utilities reducing code duplication by ~60%
- Designed metadata-driven dynamic SQL generation for business views and joins
- Implemented full CI/CD pipeline using Databricks Asset Bundles with environment promotion
- Integrated automated failure alerting using Azure Logic Apps triggered by ADF pipeline events

6. CHALLENGES SOLVED

Challenge	Solution Implemented
Handling both initial and incremental loads efficiently	Single parameterized pipeline with conditional watermark logic and if-condition activity
Avoiding empty file creation on no-new-data days	If Condition + Delete Activity to clean up empty outputs
Managing schema evolution in streaming ingestion	Autoloader with schema evolution enabled + Delta Lake schema enforcement
Implementing SCD Type 2 without complex MERGE logic	Delta Live Tables declarative CDC flows with sequence_by column
Making pipelines reusable across many tables	Metadata-driven Jinja2 templates + ForEach activity with configuration array
Secure and governed access to Data Lake from Databricks	Unity Catalog External Locations + Access Connector with managed identity
Version-controlled deployment of Databricks assets	Databricks Asset Bundles with YAML configuration and Git integration

7. SKILLS DEMONSTRATED

- End-to-End Data Pipeline Design & Architecture
- Azure Data Factory (Advanced Pipeline Development, Parameters, Expressions, Control Flow)

- Azure Databricks & PySpark (Transformations, Autoloader, Delta Lake, Serverless Compute)
- Unity Catalog & Data Governance (Metastores, External Locations, Access Control, Lineage)
- Slowly Changing Dimensions (SCD Type 2) Implementation
- Delta Live Tables / Lakeflow Declarative Pipelines
- Metadata-Driven & Dynamic Pipeline Development
- CI/CD for Data Platforms (Databricks Asset Bundles, Git Integration)
- Data Quality Enforcement & Monitoring
- Production-Grade Best Practices (Idempotency, Exactly-Once, Parameterization, Modularity)